Stacks

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time. Implement the MinStack class: MinStack() initializes the stack object. void push(int val) pushes the element val onto the stack. void pop() removes the element on the top of the stack. int top() gets the top element of the stack. int getMin() retrieves the minimum element in the stack. You must implement a solution with O(1) time complexity. for each function

var MinStack = function() {

};

/**
* @param {string} val
* @return {string}
*/
MinStack.prototype.push = function(val) {

};

/**
* @return {void}
*/
MinStack.prototype.pop = function() {

};

/**
* @return {void}
*/
MinStack.prototype.top = function() {

};

/**
* @return {void}
*/
MinStack.prototype.getMin = function() {

};

/**
* Your MinStack object will be instantiated and called as such:
* var obj = new MinStack()
* obj.push(val)
* obj.pop()
* var param_3 = obj.top()
* var param_4 = obj.getMin()
*/

"use of Javascript Stacks Method" and understand no Gujarati Language

નમસ્તે, આ સ્ટેકમેન્ટમાં સ્ટેક્સના ઘણાં પ્રકારનાં ઓપ્શન્સ અને તેમાંથી ઉપયોગ કરવાનાં પ્રકારો (push, pop, top, અને getMin સ્ટેક) ઉપલબ્ધ છે અને તે બધાં જ ટાઈમ કોમ્પ્લેક્સિટી O(1) ધરાવે છે. આ સ્ટેકમેન્ટમાં અને બીજાં સ્ટેક્સ વચ્ચે તફાવત એ છે કે તેમાં ઉપલબ્ધ કરેલાં અને કેવાયેલાં Minimum Value સ્ટેક વાપરી પણ શકાશે.

સ્ટ્રાક્ચરનું સ્પષ્ટીકરણ (Explanation)

જ્યારે અમારાથી કોઈપણ સ્ટેક બનાવવામાં આવે છે, ત્યારે અમારાથી એ બધાં જ ઓપ્શન્સ તૈયાર કરવાં પડશે. સૌથી પહેલાં અમારે એક ખાલી સ્ટેક બનાવવાનો રહેશે. ત્યારબાદ અમારે એક એવું ફંક્શન બનાવવાનું રહેશે (getMin) જેનાથી અમારાં સ્ટેકમાં સૌથી નાનાં નંબરનાં કુલ્લે કરી શકાશે. જેનાથી અમારાં સ્ટેકમાં સૌથી નાનાં નંબરનું ફંક્શન મળશે.

ઉદાહરણ: જ્યારે સ્ટેક પર કામ કરવાનું હોય ત્યારે:

1. પુશ એન્ટ્રિ (Add)
2. કંઈ સ્ટેકમાં સૌથી નાનું મૂલ્ય કેટલું છે? (Min)

Javascript Implementation :

જ્યારે Javascript ની સ્ટેકમાં કઈ રીતે અમલ કરી શકાય છે:

```javascript
var MinStack = function() {
    this.stack = [];
};

/** @param {number} val
 * @return {void}
 */
MinStack.prototype.push = function(val) {
    // જો સ્ટેક ખાલી હોય તો પહેલી વેલ્યુ ઉમેરો
    // નહીં તો, વર્તમાન વેલ્યુ અને છેલ્લે ઉમેરેલાં વેલ્યુમાં જે નાનું હોય તે ઉમેરો.
    let min = this.stack.length === 0 ? val : Math.min(val, this.getMin());
    this.stack.push([
        val,
        min
    ]);
};

/**
 * @return {void}
 */
MinStack.prototype.pop = function() {
    this.stack.pop();
};

/**
 * @return {number}
 */
MinStack.prototype.top = function() {
    return this.stack[this.stack.length - 1][0];
};

/**
 * @return {number}
 */
MinStack.prototype.getMin = function() {
    return this.stack[this.stack.length - 1][1];
};
```

૩૨ બિટ કિંમત સમ કરી શકાય (Step by Step)

ચાલો કે અમારાં પાસે નીચેનાં કમિક સ્ટેક્સ: push(10), push(20), push(30), pop()

1. push(10) : સ્ટેક [10] (કારણ કે, ત્યાં માત્ર 10 જ છે) ઉમેરાશે.
2. push(20) : સ્ટેક [10 અને 20] ઉમેરાશે. સૌથી નાનાં નંબરનાં કુલ્લે 10 અને 20 ઉમેરાશે.
3. push(30) : જો કે 3 નંબર અહીં સૌથી મોટાં નંબરનાં કુલ્લે 10 અને 20 ઉમેરાશે.
4. pop() : જો કે 3 નંબર અહીં કુલ્લે સૌથી મોટાં નંબરનાં કુલ્લે 10 અને 20 ઉમેરાશે.
5. getMin() : અહીં સૌથી નંબર મિનિમમ 10 રહેશે.
6. pop() : ઉપરાંત જો ઉમેરાયાં અને કાઢી નાંખ્યાં હોય તો સૌથી નંબર 10 જ રહેશે.

પુનરાવર્તન કરવાનું (Step by Step Explanation)

ચાલો કે અમારાં પાસે નીચેનાં કમિક સ્ટેક્સ: push(10), push(20), push(30), pop()

1. push(10) : સ્ટેક [10] ઉમેરાશે. સૌથી નાનાં નંબરનાં કુલ્લે 10 ઉમેરાશે.
2. push(20) : સ્ટેક [10 અને 20] ઉમેરાશે. સૌથી નાનાં નંબરનાં કુલ્લે 10 અને 20 ઉમેરાશે.
3. push(30) : જો કે 3 નંબર અહીં કુલ્લે સૌથી મોટાં નંબરનાં કુલ્લે 10 અને 20 ઉમેરાશે.
4. pop() : અહીં સૌથી નંબર મિનિમમ 10 રહેશે.

પુનરાવર્તન કરવાનું (Step by Step Explanation)

ચાલો કે અમારાં પાસે નીચેનાં કમિક સ્ટેક્સ: push(10), push(20), push(30), pop()

1. **પુશ ઓપરેશન:** અમારાં 3 નંબરનાં સ્ટેકમાં ઉમેરાશે.
2. **મિનિમમ વેલ્યુ અપડેટ કરવાનું:** અમારાં નંબરનાં કિંમત શું હશે (minimum) જોવાશે. (અહીં 10 નંબરનાં કુલ્લે).
3. **પોપ ઓપરેશન:** અમારાં સ્ટેકમાંથી 3 કાઢી નાંખવામાં નંબરનાં કુલ્લે આવશે. (અહીં 10 નંબરનાં કુલ્લે).
4. **અપડેટ થયેલું કુલ્લે:** અમારાં 3 નંબરનાં કુલ્લે કાઢી નાંખ્યાં પછી અમારાં 2 નંબરનાં કુલ્લે બાકી રહેશે. (અહીં 10 નંબરનાં કુલ્લે).
5. **સ્ટ્રાક્ચર જોવાનું:** અમારાં 2 નંબરનાં કુલ્લે. (અહીં 10 નંબરનાં કુલ્લે).
6. **નવું ઉમેરાયું:** અમારાં 3 નંબરનાં કુલ્લે ઉમેરાશે. (અહીં 10 નંબરનાં કુલ્લે).
7. **નવું મિનિમમ:** અમારાં નંબરનાં કિંમત શું હશે (minimum) જોવાશે. (અહીં 10 નંબરનાં કુલ્લે).

આ રીતે કેમ થાય છે?

- **Time Complexity (O(1)) :** અમારાં સ્ટેકમાંનાં બધાં જ્યારે કામ કરે અમારાં સ્ટેકમાં સૌથી નાનાં નંબરનાં કુલ્લે ઉમેરાશે.
- **Space Complexity (O(n)) :** અમારાં સ્ટેકમાં સૌથી નાનાં નંબરનાં કુલ્લે ઉમેરાશે.

શું અમારાં અહીં નંબરનાં કુલ્લે ઉમેરાય અથવા, અમારા જે અહીં કરેલાં કામ કરે છે અમારાં જુદાં?

Given two integer arrays pushed and popped each with distinct values, return true if this could have been the result of a sequence of push and pop operations on an initially empty stack, or false otherwise.

Example 1:
Input: pushed = [1,2,3,4], popped = [4,3,2,1]Output: true Explanation: We might do the following sequence: push(1), push(2), push(3), push(4), pop() → 4, push(5), pop() → 5, pop() → 3, pop() → 2, pop() → 1

var validateStackSequences = function(pushed, popped) {

};

"use of Javascript Stacks Method" and understand no Gujarati Language

આ સ્ટેકમેન્ટમાં સમજાવ્યાં મુજબ અમારાં સ્ટેક **Simulated Stack** અમારી પાસેનાં સ્ટેકમાં ઉમેરાયેલાં અમારાં pushed અને સ્ટેકમાંથી કાઢેલાં અમારાં popped અને સ્ટેકમાંનાં અમારાં બધાં જ્યારે અમારાં પોતાનાં સ્ટેકમાં ઉમેરાયેલાં અમારાં બધાં જ્યારે અમારાં પોતાનાં સ્ટેકમાંથી કાઢેલાં અમારાં બધાં સરખાં હોય અથવા અમારાં બધાં જ્યારે અમારાં પોતાનાં સ્ટેકમાં ઉમેરાયેલાં અમારાં બધાં જ્યારે અમારાં પોતાનાં સ્ટેકમાંથી કાઢેલાં અમારાં બધાં સરખાં હોય તો true અને નહીં તો false.

જો કોઈ પોપ કરી શકાતું હોય તો અમારાં સ્ટેકમાંથી અમારાં popped ઉમેરાતાં જાય.

Javascript Implementation

```javascript
var
/**
 * @param {number[]} pushed
 * @param {number[]} popped
 * @return {boolean}
 */
var validateStackSequences = function(pushed, popped) {
    let stack = [];
    let p = 0; // પોપનાં ઇન્ડેક્સ માટે વેરિએબલ
    for (let val of pushed) {
        stack.push(val); // પુશ કરેલું દરેક તત્વ ઉમેરો
        // જ્યારે સ્ટેક ખાલી ન હોય અને છેલ્લું ઉમેરેલું તત્વ
        // popped[p] જેટલું જ હોય, ત્યારે પોપ કરો
        while (stack.length > 0 && stack[stack.length - 1] === popped[p]) {
            stack.pop();
            p++; // પોઇન્ટરને આગળ ખસેડો
        }
    }
    // જો બધાં તત્વો પોપ થઈ ગયાં, તો true પરત કરો
    return stack.length === 0;
};
```

પુનરાવર્તન કરવાનું (Step by Step Explanation)

ચાલો કે અમારાં પાસે: pushed = [1, 2, 3, 4], અને popped = [4, 3, 2, 1]

1. **પુશ ઓપરેશન:** અમારાં 1, 2, અને 3 નંબરનાં અમારાં સ્ટેકમાં ઉમેરાશે.
2. **મિનિમમ વેલ્યુ અપડેટ કરવાનું:** અમારાં નંબરનાં કિંમત શું હશે (minimum) જોવાશે. (અહીં 10 નંબરનાં કુલ્લે).
3. **પોપ ઓપરેશન:** અમારાં સ્ટેકમાંથી 4 કાઢી નાંખવામાં નંબરનાં કુલ્લે આવશે. (અહીં 10 નંબરનાં કુલ્લે).
4. **અપડેટ થયેલું કુલ્લે:** અમારાં 3 નંબરનાં કુલ્લે કાઢી નાંખ્યાં પછી અમારાં 2 નંબરનાં કુલ્લે બાકી રહેશે. (અહીં 10 નંબરનાં કુલ્લે).
5. **સ્ટ્રાક્ચર જોવાનું:** અમારાં 2 નંબરનાં કુલ્લે. (અહીં 10 નંબરનાં કુલ્લે).
6. **નવું ઉમેરાયું:** અમારાં 3 નંબરનાં કુલ્લે ઉમેરાશે. (અહીં 10 નંબરનાં કુલ્લે).
7. **નવું મિનિમમ:** અમારાં નંબરનાં કિંમત શું હશે (minimum) જોવાશે. (અહીં 10 નંબરનાં કુલ્લે).

આ રીતે કેમ થાય છે?

- **Time Complexity (O(n)) :** અમારાં સ્ટેકમાંનાં બધાં જ્યારે કામ કરે અમારાં પુશ અને પોપ ઓપરેશન થાય છે.
- **Space Complexity (O(n)) :** અમારાં સ્ટેકમાં સૌથી નાનાં નંબરનાં કુલ્લે ઉમેરાશે.

શું અમારાં અહીં નંબરનાં કુલ્લે ઉમેરાય અથવા, અમારાં જે અહીં કરેલાં કામ કરે છે અમારાં જુદાં?

You are given a string s consisting of lowercase English letters. A duplicate removal consists of choosing two adjacent and equal letters, and removing them. We repeatedly make duplicate removals on s until we no longer can. Return the final string after all such duplicate removals. It can be proven that the result is unique.

Example 1:
Input: s = "abbaca"Output: "ca"Explanation: In "abbaca", we can remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca".

આ સ્ટેકમેન્ટ સ્ટ્રીંગ સાથ કરી શકાય છે. અમારાં **Stack** સ્ટ્રીંગ તરીકે કામ કરે છે. અમારાં સ્ટેક નીચે બધાં કેરેક્ટર્સ અમારાં સ્ટ્રીંગ સાથ અમારાં અમારાં સ્ટેકમાં ઉમેરાતાં જાય છે અમારાં સ્ટેકમાં જ્યારે જ્યારે એક સરખાં કેરેક્ટર્સ અમારાં સ્ટેકમાં ઉમેરાતાં અમારાં સ્ટેકમાંથી જ્યારે અમારાં પોતાનાં સ્ટેકમાં ઉમેરાતાં અમારાં પોતાનાં સ્ટેકમાંથી જ્યારે અમારાં પોતાનાં સ્ટેકમાંથી કાઢી નાંખવામાં આવે છે.

અમારાં "abbaca" સ્ટ્રીંગ માટે અમારાં સ્ટ્રીંગ સ્ટેક અમારાં અમારાં અમારાં કુલ્લે થશે.

Javascript Implementation

```javascript
var
/**
 * @param {string} s
 * @return {string}
 */
var removeDuplicates = function(s) {
    let stack = [];
    for (let char of s) {
        // જો સ્ટેક ખાલી ન હોય અને વર્તમાન અક્ષર છેલ્લા અક્ષર જેવો જ હોય
        if (stack.length > 0 && stack[stack.length - 1] === char) {
            stack.pop(); // જો પરિસ્થિતિ ટ્રુ થાય તો જૂનો અક્ષર પોપ કરો
        } else {
            stack.push(char); // અન્યથા, વર્તમાન અક્ષર ઉમેરો
        }
    }
    // છેલ્લાં બાકી રહેલાં અક્ષરોને સ્ટ્રિંગ તરીકે જોડો
    return stack.join("");
};
```

પુનરાવર્તન કરવાનું (Dry Run) "abbaca" :

ચાલો જોઈએ કે આ કોડ કઈ રીતે કામ કરે છે:

1. **Input:** "abbaca"
2. **Step 1:** 'a' સ્ટેક ખાલી છે, એટલે તેને સ્ટેકમાં ઉમેરો. સ્ટેક: ['a']
3. **Step 2:** 'b' સ્ટેકના છેલ્લા તત્વ ('a') સાથે મેચ નથી, એટલે તેને ઉમેરો. સ્ટેક: ['a', 'b']
4. **Step 3:** 'b' સ્ટેકના છેલ્લા તત્વ ('b') સાથે મેચ થાય છે, એટલે તેને પોપ કરો. સ્ટેક: ['a']
5. **Step 4:** 'a' સ્ટેકના છેલ્લા તત્વ ('a') સાથે મેચ થાય છે, એટલે તેને પોપ કરો. સ્ટેક: []
6. **Step 5:** 'c' સ્ટેક ખાલી છે, એટલે તેને ઉમેરો. સ્ટેક: ['c']
7. **Step 6:** 'a' સ્ટેકના છેલ્લા તત્વ ('c') સાથે મેચ થાય છે, એટલે તેને ઉમેરો. સ્ટેક: ['c', 'a']
8. **Final Result:** "ca"

સ્ટ્રાક્ચર સ્પષ્ટીકરણ:

- **Time Complexity (O(n)) :** અમારાં સ્ટેકમાંનાં બધાં જ્યારે કામ કરે અમારાં પુશ અને પોપ ઓપરેશન થાય.
- **Space Complexity (O(n)) :** અમારાં અમારાં સ્ટેકમાંનાં જેટલાં જેટલાં કેરેક્ટર્સ અમારાં અમારાં સ્ટેકમાં ઉમેરાશે.

આ સ્ટેકમેન્ટ સ્ટ્રીંગ સાથ કરી શકાય છે. અમારાં સ્ટ્રીંગ સાથ કેરેક્ટર્સ અમારાં અમારાં સ્ટેકમાં ઉમેરાતાં અમારાં અમારાં સ્ટેકમાંથી જ્યારે અમારાં પોતાનાં સ્ટેકમાંથી કાઢી નાંખવામાં આવે છે.

Given an encoded string, return its decoded string. The encoding rule is: k[encoded_string], where

repeated exactly k times. Note that k is guaranteed to be a positive integer.
You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k. For example, there will not be input like 3a or 2[4].
The test cases are generated so that the length of the output will never exceed 105.

Example 1:
Input: s = "3[a]2[bc]"Output: "aaabcbc"

આ સ્ટેકમેન્ટ નીચેનાં તમામ જ કામકાજ કરે છે અમારાં **Nested Structures** કોઈ દૈનિક જીવન સાથેનાં દૈનિક કોડ કરી શકે છે.

ચાલો જોઈએ કે અમારાં અમારાં સ્ટેક અમારાં સ્ટેક અમારાં સ્ટેક અમારાં કેરેક્ટર્સ અમારાં અમારાં કુલ્લે કાઢીએ.

1. **Count Stack:** દોહરાઈ રહ્યા અમારાં સ્ટ્રીંગ અમારાં કુલ્લે.
2. **String Stack:** અમારાં સ્ટ્રીંગ અમારાં સ્ટ્રીંગ.

Javascript Implementation

```javascript
var
/**
 * @param {string} s
 * @return {string}
 */
var decodeString = function(s) {
    let countStack = [];
    let stringStack = [];
    let currentString = "";
    let k = 0;

    for (let char of s) {
        if (/[0-9]/.test(char)) {
            // જો વર્તમાન અક્ષર અંક છે, તો તેનાં સાથે જોડો અને k ને અપડેટ કરો
            k = k * 10 + parseInt(char);
        } else if (char === '[') {
            // જ્યારે ઓપનિંગ બ્રેકેટ મળે, ત્યારે સ્ટેક્સમાં સાચવો
            countStack.push(k);
            stringStack.push(currentString);
            // રીસેટ કરો
            currentString = "";
            k = 0;
        } else if (char === ']') {
            // જ્યારે ક્લોઝિંગ બ્રેકેટ મળે, ત્યારે સ્ટ્રિંગ રીકન્સ્ટ્રક્ટ કરો
            let repeatCount = countStack.pop();
            let prevString = stringStack.pop();
            // વર્તમાન સ્ટ્રિંગને k વડે ગુણો અને પાછલી સ્ટ્રિંગ સાથે જોડો
            currentString = prevString + currentString.repeat(repeatCount);
        } else {
            // જો તે કોઈ અક્ષર છે, તો તેને ઉમેરો
            currentString += char;
        }
    }

    return currentString;
};
```

પુનરાવર્તન કરવાનું (Dry Run)

ચાલો જોઈએ "3[a2[bc]]" ઉદાહરણ:

1. '3' k બને છે અને 0.
2. '[': countStack માં 3 ઉમેરો અને currentString ને ખાલી કરીને પછી સ્ટેકમાં ઉમેરો. પછી, k અને currentString ખાલી થાય છે.
3. 'a': currentString માં 'a' ઉમેરો.
4. '2':
   - countStack માંથી 3 પરત આવશે.
   - stringStack માંથી "a" પરત આવશે.
   - હવે currentString = "a" + "a".repeat(3) = "aaa".
5. '[': k અને 0 બને છે.
6. 'b': currentString માં 'b' ઉમેરો. currentString: "b".
7. 'c': currentString માં 'c' ઉમેરો. હવે
8. ']':
   - countStack માંથી 2 પરત આવશે અને "bc" પરત આવશે.
   - હવે currentString = "aaa" + "bc".repeat(2) = "aaabcbc".

અમારાં અમારાં કેસ (Nested Structures):

- પહેલાં અમારાં કેસ (bc) સ્ટેકમાં અને "aaa" સ્ટ્રિંગમાં સાચવાય છે.
- પછી અમારાં કેસ (a2[bc]) સ્ટેકમાં અને "aaa" સ્ટ્રિંગમાં સાચવાય છે.

સ્ટ્રાક્ચરનું અર્થઘટન (Time Complexity): O(n), જ્યાં n એ અંતિમ સ્ટ્રિંગની લંબાઈ છે. કેમકે દરેક અક્ષરનાં **(Space Complexity):** O(n), સ્ટેકમાં કરેલાં સંગ્રહ માટે.

શું અમારાં અહીંનાં કામ કરે છે? શું અમારાં તમારાં સ્ટ્રિંગ કુલ્લે સ્ટ્રિંગ કુલ્લે? હાઈ કિંમત કરતાં કુલ્લે છે અમારાં કેવી રીતે જોઈ શકાય છે? અમારાં કુલ્લે કરેલાં અમારાં અમારાં સ્ટેકમાંનાં કુલ્લે કરેલાં **Index** કરેલાં અમારાં કુલ્લે જોઈ શકાય છે. અમારાં કુલ્લે કેવાં? અમારાં કુલ્લે કરેલાં અમારાં અમારાં સ્ટેકમાંનાં કુલ્લે કરેલાં અમારાં કુલ્લે.

અમારાં અમારાં અમારાં અમારાં તમારાં અમારાં જોઈ શકાય અહીં. તમારાં અમારાં સ્ટ્રિંગ અમારાં અમારાં અમારાં અમારાં

```
const n = name.length;
let dp = new Array(n).fill(-1);
let stack = [];// નામ અનુસારે ઇન્ડેક્સ સ્ટોર કરીશું

// મુખ્ય લૂપ: દરેક નામ માટે
for (let i = 0; i < n; i++) {
  let currentName = name[i];

  // જો આ વ્યક્તિ સ્ટેકમાં જ હોય, તો આપણે સાઈકલ શોધી કાઢીએ છીએ
  while (stack.length > 0 && name[stack[stack.length - 1]] == currentName) {
    let j = stack.pop();
    dp[j] = i - j;
  }

  // હવે આપણે આ નામને સ્ટેકમાં ઉમેરીએ છીએ
  if (i < n) {
    stack.push(i);
  }
}

return dp;
};
```

દૃશ્યમાન કામગીરી (Step-by-Step)

ચાલો કહીએ `name = ["a", "b", "a"]` છે:

1. પહેલો લૂપ (i = 0): `"a"`:
   - `if (i < 3)` સ્ટેકમાં ઉમેરો. `stack = [0]`
2. બીજો લૂપ (i = 1): `"b"`:
   - `while (stack[0] == "b")` ખોટું છે. `if (i < 3)` સ્ટેકમાં ઉમેરો. `stack = [0, 1]`
3. ત્રીજો લૂપ (i = 2): `"a"`:
   - `while (stack[1] == "a")` સાચું છે! `j = 1`. `dp[1] = 2 - 1 = 1`. `stack = [0]`
   - `if (i < 3)` સ્ટેકમાં ઉમેરો. `stack = [0, 2]`
4. અંતિમ પરિણામ (i = 3 થી): - લૂપ પૂરો થયો:
   - `[0,0] (index 0)`: આખરે જોયું છે. કોઈ અપડેટ નથી.
   - `[0,1] (index 1)`: આખરે જોયું છે. તે જોયેલ નથી. `name[1] == "a"` કરતાં અલગ છે! પરિણામ `2` થઈ જશે.
   - `[0,2] (index 2)`: આખરે જોયું છે. તે જોયેલ નથી. `name[2] == "a"` કરતાં અલગ છે! પરિણામ `2` થઈ જશે.
   - કંઈ સ્ટોરમાં કૂકર ઉમેરીએ છે (વિષય 2) આવી છે, પણ એક જોઈ શકીએ એવા જવાબ નથી.

શા માટે આ બેસ્ટ છે?

- **Time Complexity:** $O(n)$. આખરે જુઓ કે કયા ઇન્ડેક્સ સ્ટોર કરવો કંઈ કરીએ એટલિસ્ટ વધુમાં વધુ એકવાર જુઓ અને એક વાર પૂરો થાય છે.
- **Space Complexity:** $O(n)$. સ્ટોરમાં ઇન્ડેક્સ સ્ટોર કરવા માટે.

શું તમારે **Monotonic Stack** ના બીજા ઉપયોગો જોવા છે ("Daily Temperatures" સોલ્વેમ્બ) વિશે જાણવું છે? શું હું એ પણ સમજાવી શકું છું.

Given a non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1
Input height = [0,1,0,2,1,0,1,3,2,1,2,1]Output: 6Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

અહીં સોલ્વેમ (Trapping Rain Water) વ્યાપ જ પરિચિત છે. અહીં સોલ્વેમ ક્રમબદ્ધ થઈ શકતી ટેકનિક છે જેને Two Pointers કહેવાય (Dynamic Programming), પણ આપણે અહીં **Monotonic Stack** નો ઉપયોગ કરીશું કારણ કે તે વિષય નંબર ૩ સાથે વિષય નંબર ૪ સાથે સંબંધ ધરાવે છે.

સ્ટોરિક (Logic)

ટ્રેપિંગ વરસાદ છે કે પાણી ત્યારે જ ભરાય જ્યારે બંને બાજુ "ઊંચી" દીવાલ હોય. અહીં, સ્ટોરમાં રૂ. જ્યારે આપણને અચાનક જોઈ લઈએ એટલે એટલે પાણી નથી, પણ જ્યારે આપણે નીચે જઈ રહ્યા હોઈએ ત્યારે પાણી ભરાઈ જાય છે.

1. જો અચાનક જોઈતી સ્ટોરમાં ઉંચા ક્રમવાળો સ્ટોર હોય, તો તેને અમે તે ક્રમ ઉંચો જોઈ લીધો છે. અગાઉ ક્રમ ઉંચો હોય તેવાં સ્ટોરમાં પાણી ભરાઈ જશે.
2. જો અચાનક જોઈતી સ્ટોરમાં નીચા ક્રમવાળો સ્ટોર આવે છે, તો તેને અમે તે ક્રમ નીચે જઈ રહ્યા છીએ. આપણે ક્રમ ઉંચો જોઈ લીધો હોવાથી, તે પાણી ભરાઈ જશે.
3. અમારો સ્ટોરમાં ઉંચાક્રમ ઉંચો જોઈ લીધો હોવાથી, તે પાણી ભરાઈ જશે. આપણે આ પાણીનું ગણતરી કરવા માટે ઉંચાક્રમાં ઘટતો પાણીનો પ્રવાહી કરીશું.
4. કંઈ સ્ટોરમાં ક્રમ ઉંચો જોઈ લીધો છે (વિષય 2) આવી છે, પણ એક જોઈ શકીએ એવા જવાબ નથી.

શા માટે આ બેસ્ટ છે?

- **Time Complexity:** $O(n)$. આખરે જુઓ કે કયા ઇન્ડેક્સમાં સ્ટોરમાં ઉંચાક્રમ ઉંચો જોઈ લીધો છે એટલિસ્ટ વધુમાં વધુ એકવાર જુઓ અને એક વાર પૂરો થાય છે.
- **Space Complexity:** $O(n)$. સ્ટોરમાં ઇન્ડેક્સ સ્ટોર કરવા માટે.

શું તમારે **Monotonic Stack** ના બીજા ઉપયોગો જોવા છે ("Daily Temperatures" સોલ્વેમ્બ) વિશે જાણવું છે? શું હું એ પણ સમજાવી શકું છું.

Given a non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1
Input height = [0,1,0,2,1,0,1,3,2,1,2,1]Output: 6Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.

અહીં સોલ્વેમ (Trapping Rain Water) વ્યાપ જ પરિચિત છે. અહીં સોલ્વેમ ક્રમબદ્ધ થઈ શકતી ટેકનિક છે જેને Two Pointers કહેવાય (Dynamic Programming), પણ આપણે અહીં **Monotonic Stack** નો ઉપયોગ કરીશું કારણ કે તે વિષય નંબર ૩ સાથે વિષય નંબર ૪ સાથે સંબંધ ધરાવે છે.

સ્ટોરિક (Logic)

ટ્રેપિંગ વરસાદ છે કે પાણી ત્યારે જ ભરાય જ્યારે બંને બાજુ "ઊંચી" દીવાલ હોય. અહીં, સ્ટોરમાં રૂ. જ્યારે આપણને અચાનક જોઈ લઈએ એટલે એટલે પાણી નથી, પણ જ્યારે આપણે નીચે જઈ રહ્યા હોઈએ ત્યારે પાણી ભરાઈ જાય છે.

1. જો અચાનક જોઈતી સ્ટોરમાં ઉંચા ક્રમવાળો સ્ટોર હોય, તો તેને અમે તે ક્રમ ઉંચો જોઈ લીધો છે. અગાઉ ક્રમ ઉંચો હોય તેવાં સ્ટોરમાં પાણી ભરાઈ જશે.
2. જો અચાનક જોઈતી સ્ટોરમાં નીચા ક્રમવાળો સ્ટોર આવે છે, તો તેને અમે તે ક્રમ નીચે જઈ રહ્યા છીએ. આપણે ક્રમ ઉંચો જોઈ લીધો હોવાથી, તે પાણી ભરાઈ જશે.
3. અમારો સ્ટોરમાં ઉંચાક્રમ ઉંચો જોઈ લીધો હોવાથી, તે પાણી ભરાઈ જશે. આપણે આ પાણીનું ગણતરી કરવા માટે ઉંચાક્રમાં ઘટતો પાણીનો પ્રવાહી કરીશું.
4. કંઈ સ્ટોરમાં ક્રમ ઉંચો જોઈ લીધો છે (વિષય 2) આવી છે, પણ એક જોઈ શકીએ એવા જવાબ નથી.

JavaScript Implementation

```
/**
 * @param {number[]} height
 * @return {number}
 */
var trap = function(height) {
    let n = height.length;
    let waterTrapped = 0;
    let current = 0;

    while (current < height.length) {
        // જો આપણે હાલ ઊંચા ઢોળાવ પર હોય તો
        while (stack.length > 0 && height[current] > height[stack[stack.length -
            let top = stack.pop();

            if (stack.length === 0) break;// જો પાછો કોઈ ઊંચો સ્ટોર ન મળે તો પાણી

            let distance = current - stack[stack.length - 1] - 1;
            // બે સ્ટોર વચ્ચે જે અંતર છે તેટલું જ પાણી
            let boundedHeight = Math.min(height[current], height[stack[stack.leng

            waterTrapped += distance * boundedHeight;
        }
        stack.push(current++);
    }

    return waterTrapped;
};
```

દૃશ્યમાન કામગીરી (How It works)

ચાલો કહીએ `height = [2, 0, 2]` છે:

1. શરૂઆતમાં `stack = [0]` (ઇન્ડેક્સ 0)
2. જ્યારે `current = 1` (ઇન્ડેક્સ 1): (ઇન્ડેક્સ 1)
   - હવે 2 (ઇન્ડેક્સ 2) આવી છે. તે સ્ટોરમાં ઉંચા છે! અમને મળી ગયો.
   - આપણે `top = 0` (ઇન્ડેક્સ 0) ને બહાર કાઢીએ છીએ.
   - ડાબી બાજુનું ઊંચાઈ ઇન્ડેક્સ 0 પણ છે (ઇન્ડેક્સ 2).
   - જમણી બાજુનું ઊંચાઈ ઇન્ડેક્સ 2 પણ છે (ઇન્ડેક્સ 2).
   - **ગણતરી:** જથ્થો (2 - 0 - 1) × 1 = (ઊંચાઈ (min(2,2)) - 0 × 2) = **2 units પાણી.**

આ શીટ કેમ બેસ્ટ છે?

- **Time Complexity:** $O(n)$. દરેક બાર (bar) સ્ટોરમાં એકવાર જાય છે અને એકવાર બહાર નીકળે છે.
- **Space Complexity:** $O(n)$. સ્ટોર પણ જરૂરી છે જ.

આ સોલ્વેમ વિષય નં. ૩ સાથે આપણે કેટલી સંકળાયેલ છે? તમારે ઉંચાક્રમ ઉંચો જોઈ લીધો હોવાથી ઘટતો પ્રવાહી સ્ટોરમાં રાખવો પડશે કે શું?

શું હવે આ જ સોલ્વેમ **Two Pointer** ટેકનીક સમજાવવી જોઈતી? તે કેટલી ઈફેક્ટિવ? (O(1)) અવકાશ ઘટે પણ થાકી જવાનો ભય છે.

Design an algorithm that collects daily price quotes for some stock and returns the span of that stock's price for the current day.
The span of the stock's price in one day is the maximum number of consecutive days (starting from that day going backward) for which the stock price was less than or equal to the price of that day.
For example, if the prices of the stock in the last four days is [7,1,5,3] and the price of the stock today is 3, then the span of today is 4 because starting from today, the price of the stock was less than or equal 3 for 4 consecutive days.
Also, if the prices of the stock in the last four days is [7,2,1,2] and the price of the stock today is 5, then the span of today is 3 because starting from today, the price of the stock was less than or equal 5 for 3 consecutive days.
Implement the StockSpanner class:
StockSpanner() initializes the object of the class.
int next(int price) returns the span of the stock's price given that today's price is price.

Example 1
Input
["StockSpanner", "next", "next", "next", "next", "next", "next", "next"]
[[], [100], [80], [60], [70], [60], [75], [85]]Output
[null, 1, 1, 1, 2, 1, 4, 6]Explanation
StockSpanner stockSpanner = new StockSpanner();
stockSpanner.next(100); // return 1
stockSpanner.next(80); // return 1
stockSpanner.next(60); // return 1
stockSpanner.next(70); // return 2
stockSpanner.next(60); // return 1
stockSpanner.next(75); // return 4, because the last 4 prices (including today's price of 75) were less than or equal to today's price
stockSpanner.next(85); // return 6

આ એ જ પ્રશ્ન છે જે આપણે **Two Pointer** ટેકનીક સમજાવવી જોઈતી? તે કેટલી ઈફેક્ટિવ? (O(1)) અવકાશ ઘટે પણ થાકી જવાનો ભય છે.

આ એ જ પ્રશ્ન છે જે આપણે **Two Pointer** ટેકનીક સમજાવવી જોઈતી? તે કેટલી ઈફેક્ટિવ? (O(1)) અવકાશ ઘટે પણ થાકી જવાનો ભય છે.

આ સોલ્વેમ વિષય નં. ૩ સાથે સંકળાયેલ છે. આપણે સ્ટોરમાં ઉંચાક્રમ ઉંચો જોઈ લીધો હોવાથી, તે સ્ટોરમાં રાખવો પડશે કે શું? આપણે આ સોલ્વેમ **Monotonic Stack** નો ઉપયોગ કરીશું. કારણ કે તે વિષય નંબર ૩ સાથે સંબંધ ધરાવે છે.

સ્ટોરિક (Logic)

ચાલો કહીએ `price` આવે:

1. જો તે ઉંચા સ્ટોરમાં ટોચ (છેલ્લી ઉંચાક્રમ) કરતાં ઓછી કોઈક, તો તેને અમે તે ઉંચાક્રમ ઉંચો જોઈ લીધો છે. અગાઉ ક્રમ ઉંચો હોય તેવાં સ્ટોરમાં પાણી ભરાઈ જશે.
2. જો અચાનક જોઈતી નીચા ક્રમવાળો સ્ટોર આવે છે, તો તેને અમે તે ક્રમ નીચે જઈ રહ્યા છીએ. આપણે ક્રમ ઉંચો જોઈ લીધો હોવાથી, તે પાણી ભરાઈ જશે.
3. અમારો સ્ટોરમાં ઉંચાક્રમ ઉંચો જોઈ લીધો હોવાથી, તે પાણી ભરાઈ જશે. આપણે આ પાણીનું ગણતરી કરવા માટે ઉંચાક્રમાં ઘટતો પાણીનો પ્રવાહી કરીશું.
4. કંઈ સ્ટોરમાં ક્રમ ઉંચો જોઈ લીધો છે (વિષય 2) આવી છે, પણ એક જોઈ શકીએ એવા જવાબ નથી.

JavaScript Implementation

```
var StockSpanner = function() {
    // હવે આપણે ખાલીમાં સ્ટોર કરીશું: [price, span]
    this.stack = [];
};

/** @param {number} price
 * @return {number}
 */
StockSpanner.prototype.next = function(price) {
    let currentSpan = 1;

    // આ લૂપ જૂનો સ્ટોર ચેક કરે છે અને જરૂરીયાત મુજબ પાછળ જઈને સ્પાન કરીશું
    while (this.stack.length > 0 && this.stack[this.stack.length - 1].price <= pr
        // જૂનો સ્ટોરમાંથી નાના સ્પાનને આપણે ઉમેરી લઈ રહ્યા છીએ
        currentSpan += this.stack.pop().span;
    }

    // નવાયમાં કરીશું અને બધાનો કરીશું જેથી ભવિષ્ય લૂપ ઝડપી બને
    this.stack.push({
        price: price,
        span: currentSpan
    });

    return currentSpan;
};

/** * Your StockSpanner object will be instantiated and called as such:
 * var obj = new StockSpanner()
 * var param_1 = obj.next(price)
 */
```

દૃશ્યમાન કામગીરી (Dry Run)

ચાલો કહીએ આપણે આવી છે: `[100, 80, 60, 70]`

1. `next(100)`: સ્ટોર ખાલી છે. `stack = [{price: 100, span: 1}]`. રિટર્ન 1.
2. `next(80)`: `80 <= 100` ખોટું (ખોટું). `currentSpan = 1`. સ્ટોર `[{price: 100, span: 1}, {price: 80, span: 1}]`. રિટર્ન 1.
3. `next(60)`: `60 <= 80` ખોટું. `currentSpan = 1`. સ્ટોર `[{price: 100, span: 1}, {price: 80, span: 1}, {price: 60, span: 1}]`. રિટર્ન 1.
4. `next(70)`:
   - `70 >= 60` (સાચું) `currentSpan = 1 + 1` (છેલ્લો ઉંચાક્રમ) = 2.
   - `70 >= 80` (ખોટું).
   - સ્ટોર `[{price: 100, span: 1}, {price: 80, span: 1}, {price: 70, span: 2}]`. રિટર્ન 2.

દૃશ્ય ફાયદા (Key Benefits)

- **Time Complexity:** $O(1)$ સરેરાશ (Amortized). દરેક ઉંચાક્રમ ફક્ત એક જ વાર ઘટતા છે અને એક જ વાર ઉમેરાય છે.
- **Space Complexity:** $O(n)$ સ્ટોરમાં. $O(n)$ સ્ટોરમાં.

આ સોલ્વેમમાં અમારાથી પ્રશ્નોના અમારાથી તમને અમારાથી તમને `[8, 10, 9, 12]` જો `[8, 10]` કરતા મોટો હોય, તો જરૂરિયાત જે કંઈ ઉંચાક્રમ ઉંચો કરતા મોટો અમારો, તો તે અમારાથી તમને `10` કરતા તો મોટો કંઈ જ નહોતો. એટલે અમારાથી `10` ને અમારાથી ઉંચા કરવાની જરૂર કંઈ નથી.

શું હવે ઈમોજી રોકે કે અમને આવું કરવાનું ટ્રિક્સ કુલ જ્યાં સ્ટાર ટ્રેકી કરવાનું?

Largest Rectangle in Histogram

Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Example 1
Input: heights = [2,1,5,6,2,3]Output: 10Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.

Analysis

આ સોલ્વેમ 'Largest Rectangle in Histogram' તરીકે ઓળખાય છે અને તે ટ્રેડિંગમાં મદદ **Monotonic Increasing Stack** તકનીક અસરકારક રીત છે.

અહીં અમારાથી ઉંચામાં ઉંચી લાઈન કઈ છે ઉંચાઈ `[2]` અને `1` વચ્ચે તો બાકી બધા ઉંચા અંતર `[2]` અંતરમાં અમારાથી ઉંચાઈમાં 2 અને મધ્યમાં ઉંચાઈમાં 1.

સ્ટોરિક (Logic)

ચાલો અમારાથી ઉંચાઈઓનાં અંતરમાં અંતરમાં.

1. જો અમારાં ભવિષ્યની ઉંચાઈમાં સ્ટોરમાં ઉંચા ક્રમવાળો સ્ટોર હોય, તો તેને અમે તે ક્રમ ઉંચો જોઈ લીધો છે.
2. જો અમારાં ભવિષ્યની ઉંચાઈમાં સ્ટોરમાં ઉંચા ક્રમવાળો સ્ટોર હોય, તો તેને અમે તે ક્રમ નીચે જઈ રહ્યા છીએ. અમારાં ઉંચાક્રમ ઉંચાક્રમ ઉંચાક્રમ (rectangle) અમારાથી ભવિષ્યની ઉંચાઈમાં સ્ટોરમાં.
3. અમારાં ઉંચાક્રમાં નવા ક્રમવાળો સ્ટોર ટોચ પર આવતાં અમારાં ઉંચાક્રમ ઉંચાક્રમ.
   - **Height:** સ્ટોરમાંથી મળેલો ક્રમવાળો ઉંચાક્રમ.
   - **Width:** અમારાથી તે ઉંચાક્રમ અને સ્ટોરમાં રાખેલ નવા ઇન્ડેક્સ વચ્ચેનું અંતર.

JavaScript Implementation

```
/**
 * @param {number[]} heights
 * @return {number}
 */
var largestRectangleArea = function(heights) {
    let n = heights.length; // કુલ બાર કેટલા છે તે જાણો
    let maxArea = 0;

    // સ્ટેક માટે એ ઇન્ડેક્સનો સ્ટોરમાં રાખવાનો જ્યાં બાર પાછા ઉમેરી શકીએ જો જરૂર
    heights.push(0);

    for (let i = 0; i < heights.length; i++) {
        // જ્યારે કોઈ બારનું ઉંચાઈ નક્કી થઈ ગયું, ત્યારે તેનું ક્ષેત્રફળ ગણો
        while (stack.length > 0 && heights[i] < heights[stack[stack.length - 1]]
            let h = heights[stack.pop()]; // ઉંચાઈ
            // જો કોઈ જગ્યાએ ઉંચાઈ તે પહોચતી (i - ક્યાંક, ઉદાહરણ (i - stackTop - 1)
            let w = stack.length === 0 ? i : i - stack[stack.length - 1] - 1;
            maxArea = Math.max(maxArea, h * w);
        }
        stack.push(i);
    }

    // છેલ્લે કંઈ બાકી રહ્યું હોય તો તેનો ક્ષેત્રફળ ગણો
    heights.pop();

    return maxArea;
};
```

દૃશ્યમાન કામગીરી (Dry Run): [2, 1, 5, 6, 2, 3, 10]

1. Index 0 (height 2): સ્ટોરમાં ખાલી. `stack = [0]`
2. **Index 1 (height 1):** 2 > 1.
   - pop 0 (height 2): એરિયા: 2 × 1 = 2.
   - `stack = [1]`
3. **Index 2 & 3 (Height 5, 6):** બંનેનાં ઉંચાઈ વધી રહી છે, એટલે સ્ટોરમાં ઉમેરો. `stack = [1, 2, 3]`
4. **Index 4 (height 2):** 6 > 2.
   - pop 3 (height 6): એરિયા: 6 × (4 − 2 − 1) = 6 × 1 = 6.
   - હજી 2 > 5 છે. pop 2 (height 5): એરિયા: 5 × (4 − 1 − 1) = 5 × 2 = **10.**
   - અંતમાં એરિયા 10 થઈ ગયો.
5. આ પ્રક્રિયા અંત સુધી ચાલતી જાય છે (છેલ્લે સ્ટોરી પરિણામ 10 આવશે).

આ શીટ કેમ બેસ્ટ છે?

- **Time Complexity:** $O(n)$ - દરેક બાર ફક્ત એકવાર સ્ટેક પર પાછા ઉમેરી અને એક જ વાર બહાર નીકળે છે.
- **Space Complexity:** $O(n)$ - સ્ટેક પણ.

શું તમારે આ સોલ્વેમમાં મદદ કરવા માટે મારાથી કોઈ ડાયાગ્રામ (Histogram) અથવા ચાર્ટ બનાવી શકું? (Matrix) સમજવાની રીત? શું તમને કોઈ બાકી પ્રશ્ન છે?

Gemini can make mistakes, so double-check it